

Tokenization

Deep Dive

abc

1 2
3 4

How text becomes numbers — how tokenizers are trained, and why the same sentence costs 3× more in some languages.

"unhappiness"



"un"

"happi"

"ness"



359

8102

1467



! THE PROBLEM

The model can't read letters, and it can't memorize every word

Neural networks operate on numbers. How you split text into those numbers is a hard design problem, and the two obvious approaches both fail.

Option A: ONE TOKEN PER WORD

Vocab needs every word: "run", "running", "runner"
+ every typo, name, and new coinage. Millions of entries.
Unknown words fail completely (Out of Vocabulary).

Option B: ONE TOKEN PER CHARACTER

Tiny vocab (100 symbols).
But "tokenization" becomes 12 tokens. Sequences get hugely long.
Attention cost explodes quadratically.

The solution (Subword Tokenization): Split text into pieces between characters and words. Common words stay whole; rare words break into fragments.

WORD-LEVEL

["running"]

millions vocab X

CHAR-LEVEL

[r] [u] [n] [n] [i] [n]
[g]

too long X

SUBWORD (Now)

["run"] ["ning"]

~50k vocab ✓



AI Coach John



Follow

↻ Repost

The Full Pipeline: Text → Numbers

1. NORMALIZATION

```
"Hélló  World!" → "hélló world!"  
(lowercase, strip whitespace)
```

2. PRE-TOKENIZATION

```
→ ["héllö", " world", "!"]  
  (initial split on whitespace)
```

3. TOKENIZATION

```
→ ["hé", "llo", " world", "!"]  
(apply learned merge rules)
```

4. NUMERICALIZATION

```
→ [3245, 8891, 1102, 0]
    (lookup in vocab dictionary)
```

5. EMBEDDING LOOKUP

```
→ [[0.2, -0.4, ...], [0.8, ...]]  
(ID indexes embedding matrix)
```

into the model

The key realization: Step 4 is a pure dictionary lookup. Step 5 is where numbers become meaningful vectors (learned during training). The tokenizer itself contains **no semantics**.



AI Coach John



Follow

 **Repost**

How the Vocabulary Maps to Numbers: **It's just a dictionary**

```
# A tokenizer's vocabulary is literally a dict
vocab = {
    "<pad>": 0,
    "<unk>": 1,
    "<s>": 2,      # special tokens usually lowest
    "the": 464,
    "cat": 3797,
    "ization": 1634,
    "_un": 359,
}

# Encoding = lookup ( "cat" → 3797 )
# Decoding = reverse lookup ( 3797 → "cat" )
```

- ✓ **IDs are arbitrary** — 3797 has no numeric relationship to 3798. All meaning lives in the embedding vector.
- ✓ **Frequency ordering** — more frequent tokens generally get lower IDs (just an artifact of how it's built).



Byte Pair Encoding (BPE): Merging frequent pairs

BPE isn't hand-designed — it's *learned* from a corpus. The algorithm is remarkably simple:

START: split everything into characters

"low low lower newest widest"

→ l,o,w l,o,w l,o,w,e,r n,e,w,e,s,t w,i,d,e,s,t

STEP 1: merge most frequent pair ("l"+"o" appears 3×)

→ lo,w lo,w lo,w,e,r n,e,w,e,s,t w,i,d,e,s,t

STEP 2: repeat ("lo"+"w" appears 3×)

→ low low low,e,r n,e,w,e,s,t w,i,d,e,s,t

STEP 3: repeat ("e"+"s" appears 2×)

→ low low low,e,r n,e,w,es,t w,i,d,es,t

... continue until vocabulary size hit (e.g. 50,000)

What gets saved: The ordered list of merge rules. At inference, the tokenizer replays those exact same merges on new text.



AI Coach John



Follow

 Repost

Why BPE Works: **Accidental morphology**

Nobody told BPE about prefixes/suffixes. But because "ing", "un", and "ness" appear frequently, they get merged early and become their own tokens.

```
"unhappiness" → ["un"]["happi"]["ness"]  
"tokenization" → ["token"]["ization"]  
"running" → ["runn"]["ing"]
```

- ✓ Unseen words decompose into familiar pieces
- ✓ Related words share tokens ("happiness" / "unhappiness")

THE FREQUENCY EFFECT

"the"		1 token
"tokenization"	 	2 tokens
"Kadambathur"	    	5+ tokens



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

WordPiece (BERT): Different merge criterion

Works almost identically to BPE, but differs in *which* pair it merges.

BPE: merge pair that appears MOST FREQUENTLY

WordPiece: merge pair that MOST INCREASES likelihood
 $\text{score} = \text{freq}(\text{AB}) / (\text{freq}(\text{A}) \times \text{freq}(\text{B}))$

BPE happily merges common characters. WordPiece penalizes pairs if the pieces are common individually—favoring pairs that *genuinely belong together*.

THE VISIBLE SIGNATURE:

BPE: "playing" → ["play"]["ing"]

WordPiece: "playing" → ["play"]["##ing"]

↑ "##" marks continuation of the same word



AI Coach John



Follow

↻ Repost

SentencePiece: Solving the whitespace problem

BPE and WordPiece assume you can pre-split on whitespace. This breaks for a large share of the world's languages.

English:	"the cat sat"	→ spaces mark boundaries ✓
Chinese:	"我喜欢吃苹果"	→ no spaces at all ✗
Japanese:	"私は学生です"	→ no spaces at all ✗
Thai:	"ฉันชอบกินแอปเปิ้ล"	→ no spaces at all ✗

Solution: Treat text as a raw stream of Unicode characters. Spaces become an actual character: _

BPE/WordPiece: [split on spaces] → ✗ fails on Thai/Chinese

SentencePiece: [raw unicode] → ✓ works everywhere

"the cat" → ["_the", "_cat"]



AI Coach John



Follow

↻ Repost

The Three Compared: BPE vs WordPiece vs SentencePiece

	BPE	WordPiece	SentencePiece
Merge criteria	High frequency	Likelihood gain	Wraps BPE/Unigram
Needs spaces?	Yes	Yes	No — raw text
Space handling	Discarded	Discarded	Encoded as _
Continuation	None / Ġ	## prefix	_ marks start
Used by	GPT, Llama	BERT	T5, Multilingual



SentencePiece isn't a competing algorithm—it's a framework that handles the raw-text layer. "SentencePiece BPE" is extremely common.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Byte-Level BPE: What most LLMs actually use

Modern GPT-family tokenizers run BPE over raw *bytes* rather than Unicode characters. Base vocabulary = 256 possible byte values.

Zero out-of-vocabulary cases, ever.
Any text, any emoji, any corrupted input
can ALWAYS be represented.

The cost: Non-Latin scripts pay heavily. A character outside ASCII takes multiple UTF-8 bytes, consuming multiple tokens.

English "a"	1 byte	part of 1 token
Emoji "🌈"	4 bytes	multiple tokens
Hindi "क"	3 bytes	multiple tokens

⚠️ Guaranteed coverage ✓, but uneven cost ⚠️



AI Coach John



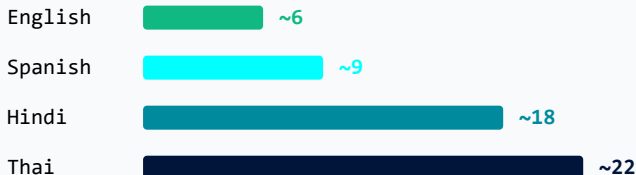
Follow

↻ Repost

Why Costs Differ: Same meaning, different price

Three compounding causes: English-dominated training data, UTF-8 byte costs for non-Latin scripts, and morphologically rich languages splitting into more fragments.

"I LOVE LEARNING NEW THINGS"



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 **Repost**

The Real Consequences: It costs money and capability

- × **Cost inequality** — APIs bill per token.
- × **Latency increases** — more tokens = slower generation.
- × **Quality suffers** — highly fragmented words are harder for the model to represent coherently.

EFFECTIVE CONTEXT SHRINKS

96K words

English

30K words

Hindi

Same 128K token window. Radically different content capacity.



AI Coach John



Follow

 Repost

Why Models Fail At trivially easy things

"strawberry" as the model sees it:

["str"] ["aw"] ["berry"]



No individual letters are visible.

"Count the r's" is a genuinely hard task!

Arithmetic inconsistency

"1234" → ["123"]["4"] or ["12"]
["34"]. Positions misalign.

Whitespace sensitivity

"hello" (15339) vs " hello"
(24748). Leading space =
different token.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Working With Tokenizers: Inspect what's happening

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("gpt2")

text = "Tokenization is fascinating!"

# 1. See the pieces
print(tok.tokenize(text))
# ['Token', 'ization', 'Ġis', 'Ġfascinating', '!']
#           ▲ Ġ represents a space

# 2. Get the IDs and length
ids = tok.encode(text) # [30642, 1634, 318, 13899, 0]
print(len(ids))        # 5

# 3. Always test your languages!
for t in ["I love learning", "मुझे सीखना पसंद है"]:
    print(len(tok.encode(t)), "tokens")
```



Rules That Prevent Silent failures

⚠ Never mix them

Using the wrong tokenizer indexes wrong embedding rows. Outputs nonsense silently.

⚠ Token $\neq$ word

Ratio varies heavily by language and type (code vs prose).

⚠ Count with the real one

Heuristics are for sanity checks, not for cost or context budgeting.

⚠ Chunk by tokens

A 500-character string can silently exceed an embedding model's limit.

⚠ Special tokens cost

Chat template markers and padding count against your budget.

⚠ Whitespace matters

Leading space changes tokenization and can shift output quality.



AI Coach John



Follow

 Repost